

Unit-7: I/O Management and Generics

Unit-7: Topics and Subtopics Table

Main Topic	Subtopics	What You Learn
1. I/O Management (File Handling Basics)	File concept	What is file in Java
	Stream concept	Input & Output streams
	Byte stream vs Character stream	Data handling types
	File class	File handling operations
2. File Handling in Java	Creating file	File creation methods
	Reading file	FileReader, BufferedReader
	Writing file	FileWriter, BufferedWriter
	File properties	Name, size, path
3. Random Access File	Concept	Access any position in file
	read(), write(), seek()	Direct file operations
	Difference vs sequential access	Efficiency comparison
4. Exception Handling in File I/O	IOException	File-related errors
	FileNotFoundException	File missing case
	try-catch-finally in file handling	Safe execution
5. Generics Concept	Purpose of Generics	Type safety
	Problem without generics	Runtime errors
	Advantages of generics	Reusability, safety
6. Defining Generic Classes	Generic class syntax	<T> usage
	Type parameter	Placeholder type
	Multiple type parameters	<K, V>
7. Generic Methods	Defining generic methods	Method-level generics
	Type inference	Automatic type detection
8. Generics with Collections	ArrayList<T>	Type-safe list
	HashMap<K, V>	Key-value storage
	LinkedList<T>	Dynamic structure
9. Real-World Applications	File processing systems	Log files, reports
	Data storage systems	Databases, records
	Generics usage	Reusable libraries

Unit-7: I/O Management (File Handling Basics)

This section explains **how Java programs interact with external data (files, keyboard, network)**. It is the base for all file operations in Java.

1. File Concept (What is File in Java)

Definition: A file is a collection of data stored permanently on storage devices (HDD/SSD).

Deep Understanding

- Variables store data **temporarily (RAM)**
- Files store data **permanently (Disk)**

RAM → temporary → lost after program ends

File → permanent → saved for future use

Real Examples

student.txt → student records

data.csv → dataset

image.jpg → image file

log.txt → application logs

Why Files are Important

Purpose	Explanation
Data storage	Save data permanently
Data sharing	Transfer between programs
Data processing	Read/write large data
Logging	Store system activities

Java Perspective

Java does NOT directly deal with disk hardware.

Instead, it uses **classes and streams** to interact with files.

2. Stream Concept (Input & Output Streams)

Definition: A stream is a flow of data between a program and a source/destination.

Simple Understanding: Stream = Data Pipeline

Types of Streams

Type	Direction
Input Stream	Data comes INTO program
Output Stream	Data goes OUT of program

Diagram: File / Keyboard → Input Stream → Java Program → Output Stream → File / Screen

Example

Input Stream: Reading data from file

Output Stream: Writing data to file

Java Classes

Stream Type	Example Class
Input	FileInputStream, FileReader

Output	FileOutputStream, FileWriter
---------------	------------------------------

Key Insight

Streams allow program to communicate with external world

3. Byte Stream vs Character Stream

Java handles data in two main formats:

Byte Stream

Concept: Handles **raw binary data (0s and 1s)**.

Classes

- FileInputStream
- FileOutputStream

Use Cases

Data Type	Example
Images	.jpg, .png
Audio	.mp3
Video	.mp4
Binary files	any non-text

Example

```
import java.io.*;
class Test {
    public static void main(String[] args) throws Exception {
        FileInputStream f=new FileInputStream("data.txt");
        int i;
        while((i=f.read())!=-1) {
            System.out.print((char)i);
        }
        f.close();
    }
}
```

Character Stream

Concept: Handles **text data (characters)**.

Classes

- FileReader
- FileWriter

Use Cases

Data Type	Example
Text files	.txt
CSV files	.csv
Logs	.log

Example

```
import java.io.*;
class Test {
    public static void main(String[] args) throws Exception {
        FileReader f=new FileReader("data.txt");
        int i;
        while((i=f.read())!=-1) {
            System.out.print((char)i);
        }
        f.close();
    }
}
```

Difference: Byte vs Character Stream

Feature	Byte Stream	Character Stream
Data type	Binary	Text
Unit	Byte (8-bit)	Character (16-bit)
Classes	FileInputStream	FileReader
Use	Images, audio	Text files
Encoding	No	Yes (Unicode)

Key Insight

Byte stream → raw data

Character stream → human-readable text

4. File Class (java.io.File)

Concept: The **File** class represents a file or directory path in Java.

☞ It does NOT read/write data

☞ It only provides **information and operations on file**

Creating File Object

File f=new File("data.txt");

Important Methods

Method	Purpose
createNewFile()	Create file
exists()	Check file exists
delete()	Delete file
length()	File size
getName()	File name
getAbsolutePath()	Full path
canRead()	Check readable
canWrite()	Check writable

Example

```
import java.io.*;
class Test {
    public static void main(String[] args) throws Exception {
        File f=new File("data.txt");
```

```

        if(f.createNewFile()) {
            System.out.println("File created");
        } else {
            System.out.println("File already exists");
        }
        System.out.println("File name: "+f.getName());
        System.out.println("Path: "+f.getAbsolutePath());
        System.out.println("Size: "+f.length());
    }
}

```

Output Example

File created
 File name: data.txt
 Path: C:\...
 Size: 0

Key Understanding

File class → manage file (create, delete, info)
 Streams → read/write file data

5. Real-World Understanding

Example: Student Management System

Student data stored in file (students.txt)

Operations:

- Read student records → Input stream
- Add new student → Output stream
- Check file details → File class

Example: Log System

App writes logs into log.txt

- Output stream → write logs
- File class → check size, existence

Unit-7: File Handling in Java

File handling is used to **create, read, write, and manage files** in Java.
 It allows programs to work with **permanent data stored on disk**.

1. Creating File (File Creation Methods)

Concept: Java uses the **File class** to create and manage files.

Method: createNewFile()

Creates a new file if it does not already exist.

Example

```

import java.io.*;
class Test {
    public static void main(String[] args) throws Exception {
        File f=new File("data.txt");
    }
}

```

```

        if(f.createNewFile()) {
            System.out.println("File created");
        } else {
            System.out.println("File already exists");
        }
    }
}

```

Output

File created

Important Points

Feature	Explanation
Returns boolean	true if created
Creates empty file	Yes
Throws exception	IOException

File Creation Flow

Program → File object → createNewFile() → File created on disk

2. Reading File

Reading means **taking data from file into program.**

Using FileReader

Concept: Used to read **character (text) data** from file.

Example

```

import java.io.*;
class Test {
    public static void main(String[] args) throws Exception {
        FileReader fr=new FileReader("data.txt");
        int i;
        while((i=fr.read())!=-1) {
            System.out.print((char)i);
        }
        fr.close();
    }
}

```

How It Works

read() → reads one character → returns ASCII value → convert to char

Limitation

- Reads **one character at a time**
- Slow for large files

Using BufferedReader

Concept: Used to read **text efficiently (line by line).**

Example

```

import java.io.*;
class Test {

```

```

public static void main(String[] args) throws Exception {
    BufferedReader br=new BufferedReader(new FileReader("data.txt"));
    String line;
    while((line=br.readLine())!=null) {
        System.out.println(line);
    }
    br.close();
}
}

```

Advantages

Feature	Explanation
Faster	Uses buffer
Reads line by line	Efficient
Better performance	For large files

Key Understanding

FileReader → basic reading

BufferedReader → fast reading

3. Writing File

Writing means **storing data from program into file.**

Using FileWriter

Concept: Used to write **text data into file.**

Example

```

import java.io.*;
class Test {
    public static void main(String[] args) throws Exception {
        FileWriter fw=new FileWriter("data.txt");
        fw.write("Hello Java");
        fw.close();
    }
}

```

Important Points

Feature	Explanation
Overwrites file	Yes (default)
Writes text	Yes
Must close file	Yes

Append Mode

FileWriter fw=new FileWriter("data.txt", true);

🔗 Adds data instead of replacing

Using BufferedWriter

Concept: Used to write data **efficiently using buffer.**

Example

```

import java.io.*;
class Test {
    public static void main(String[] args) throws Exception {
        BufferedWriter bw=new BufferedWriter(new FileWriter("data.txt"));
        bw.write("Hello Java");
        bw.newLine();
        bw.write("File Handling");
        bw.close();
    }
}

```

Advantages

Feature	Explanation
Faster writing	Yes
Efficient	Buffering
Supports newLine()	Yes

Key Understanding

FileWriter → simple writing

BufferedWriter → efficient writing

4. File Properties

Using File class, we can get **information about file**.

Example

```

import java.io.*;
class Test {
    public static void main(String[] args) {
        File f=new File("data.txt");
        System.out.println("File name: "+f.getName());
        System.out.println("Path: "+f.getAbsolutePath());
        System.out.println("Size: "+f.length());
        System.out.println("Exists: "+f.exists());
        System.out.println("Readable: "+f.canRead());
        System.out.println("Writable: "+f.canWrite());
    }
}

```

Output Example

File name: data.txt

Path: C:\...

Size: 25

Exists: true

Readable: true

Writable: true

Important Methods

Method	Purpose
getName()	File name

getAbsolutePath()	Full path
length()	File size
exists()	Check existence
canRead()	Read permission
canWrite()	Write permission
delete()	Delete file

5. Complete Example (Create + Write + Read)

```
import java.io.*;

class Test {
    public static void main(String[] args) throws Exception {
        File f=new File("data.txt");
        f.createNewFile();
        FileWriter fw=new FileWriter(f);
        fw.write("Hello Java\nFile Handling");
        fw.close();
        BufferedReader br=new BufferedReader(new FileReader(f));
        String line;
        while((line=br.readLine())!=null) {
            System.out.println(line);
        }
        br.close();
    }
}
```

Execution Flow

Create file → Write data → Read data → Display output

6. Real-World Example

Student Record System

students.txt

Operations:

- Add student → FileWriter
- Read records → BufferedReader
- Check file → File class

Logging System

log.txt

- Write logs → BufferedWriter
- Read logs → BufferedReader
- Check size → File.length()

Final Understanding

File creation → createNewFile()

Reading → FileReader / BufferedReader

Writing → FileWriter / BufferedWriter

Properties → File class methods

Goal:

- ✓ Store permanent data
- ✓ Read data efficiently
- ✓ Write data safely
- ✓ Manage file information

Unit-7: Random Access File

Random access allows a program to **read or write data at any position in a file directly**, without processing the file from the beginning.

1. Random Access File Concept

Definition: A **Random Access File (RAF)** allows **non-sequential (direct) access** to any location in a file.

Deep Understanding

In normal file handling:

Start → Read → Next → Next → ... (sequential)

In random access:

Jump directly to position → Read/Write

Java Class Used

`java.io.RandomAccessFile`

Key Idea

File = array of bytes

Each byte has position (index)

👉 Random access works using **file pointer (cursor)**

File Pointer

- Points to current position in file
- Moves automatically after read/write
- Can be changed using `seek()`

2. Creating RandomAccessFile

Syntax

```
RandomAccessFile raf=new RandomAccessFile("file.txt","mode");
```

Modes

Mode	Meaning
"r"	Read only
"rw"	Read + Write

Example

```
import java.io.*;
class Test {
    public static void main(String[] args) throws Exception {
        RandomAccessFile raf=new RandomAccessFile("data.txt","rw");
        raf.writeBytes("Hello Java");
        raf.close();
    }
}
```

```
}
```

3. read() Method

Concept: Used to read data from file starting from current pointer position.

Example

```
import java.io.*;
class Test {
    public static void main(String[] args) throws Exception {
        RandomAccessFile raf=new RandomAccessFile("data.txt","r");
        int i;
        while((i=raf.read())!=-1) {
            System.out.print((char)i);
        }
        raf.close();
    }
}
```

How It Works

read() → reads 1 byte → moves pointer forward

Key Points

Feature	Explanation
Reads byte-by-byte	Yes
Returns int	ASCII value
Moves pointer	Yes

4. write() Method

Concept: Used to write data into file at current pointer position.

Example

```
import java.io.*;
class Test {
    public static void main(String[] args) throws Exception {
        RandomAccessFile raf=new RandomAccessFile("data.txt","rw");
        raf.writeBytes("ABC");
        raf.close();
    }
}
```

Important Behavior

- Writes data at pointer location
- Overwrites existing content if pointer not at end

Key Points

Feature	Explanation
Writes data	Yes
Pointer moves forward	Yes
Can overwrite	Yes

5. seek() Method (Most Important)

Concept: Used to move file pointer to a specific position.

Syntax: raf.seek(position);

Example

```
import java.io.*;
class Test {
    public static void main(String[] args) throws Exception {
        RandomAccessFile raf=new RandomAccessFile("data.txt","rw");
        raf.writeBytes("Hello World");
        raf.seek(6);
        raf.writeBytes("Java");
        raf.close();
    }
}
```

Output in File

Hello Java

How It Works

Hello World

↑

position 6 → overwrite with "Java"

Key Points

Feature	Explanation
Moves pointer	Yes
Allows direct access	Yes
Works in bytes	Yes

6. Sequential vs Random Access

Sequential Access

Concept: Data is accessed **in order (start to end)**.

Flow: Start → Read → Next → Next → ...

Example: FileReader, BufferedReader

Random Access

Concept: Data can be accessed **directly at any position**.

Flow

Jump to position → Read/Write

Difference Table

Feature	Sequential Access	Random Access
Access type	Linear	Direct
Speed	Slower for large files	Faster
Flexibility	Low	High
Pointer control	No	Yes
Example	FileReader	RandomAccessFile

Key Insight

Sequential → Read everything step-by-step

Random → Jump directly to needed data

7. Complete Example (Read + Write + Seek)

```
import java.io.*;
class Test {
    public static void main(String[] args) throws Exception {
        RandomAccessFile raf=new RandomAccessFile("data.txt","rw");
        raf.writeBytes("ABCDEFGH");
        raf.seek(3);
        raf.writeBytes("XYZ");
        raf.seek(0);
        int i;
        while((i=raf.read())!=-1) {
            System.out.print((char)i);
        }
        raf.close();
    }
}
```

Output

ABCXYZGH

8. Real-World Examples

1. Database Systems

Directly access record using index

2. Video Streaming

Jump to specific time in video

3. File Editing Software

Edit specific part without reading whole file

4. Banking System

Update specific account record

Final Understanding

RandomAccessFile:

- ✓ Direct access to file
- ✓ Uses file pointer
- ✓ seek() controls position
- ✓ read() reads data
- ✓ write() writes data

Sequential → step-by-step

Random → jump anywhere

Unit-7: Exception Handling in File I/O

File operations involve **external resources (disk, files, permissions)**.

Because of this, errors are common and must be handled properly to avoid program crashes.

1. Why Exception Handling is Needed in File I/O

Core Idea: File operation → depends on system → may fail → exception occurs

Common Problems

- File does not exist
- No permission to access file
- File is already in use
- Disk issues

☞ These are **runtime problems**, so we use **exception handling**

2. IOException

Definition: IOException is a **general exception for input/output operations**.

☞ It is a **checked exception**, so it must be handled.

When It Occurs

Situation	Example
File read/write fails	Disk error
Stream error	Broken stream
File operation fails	Permission issue

Example

```
import java.io.*;
class Test {
    public static void main(String[] args) {
        try {
            FileReader fr=new FileReader("data.txt");
            fr.close();
        } catch(IOException e) {
            System.out.println("I/O Error occurred");
        }
    }
}
```

Key Understanding

IOException = general file handling error

3. FileNotFoundException

Definition: FileNotFoundException occurs when a file is **not found or cannot be opened**.

Type

- It is a **subclass of IOException**

When It Occurs

Situation	Example
File does not exist	Wrong file name
Wrong path	Incorrect directory

No access permission	Restricted file
----------------------	-----------------

Example

```
import java.io.*;
class Test {
    public static void main(String[] args) {
        try {
            FileReader fr=new FileReader("abc.txt");
        } catch(FileNotFoundException e) {
            System.out.println("File not found");
        }
    }
}
```

Key Understanding

FileNotFoundException = specific case of IOException

4. try-catch-finally in File Handling

This is the **standard way to safely handle file operations**.

try Block

Concept: Contains code that may cause exception.

Example

```
try {
    FileReader fr=new FileReader("data.txt");
}
```

catch Block

Concept: Handles the exception when it occurs.

Example

```
catch(IOException e) {
    System.out.println("Error handling file");
}
```

finally Block

Concept: Executes **always**, whether exception occurs or not.

Purpose

- Close file
- Release resources

Example

```
finally {
    System.out.println("Operation complete");
}
```

Complete Example

```
import java.io.*;
```

```

class Test {
    public static void main(String[] args) {
        FileReader fr=null;
        try {
            fr=new FileReader("data.txt");
            int i;
            while((i=fr.read())!=-1) {
                System.out.print((char)i);
            }
        } catch(FileNotFoundException e) {
            System.out.println("File not found");
        } catch(IOException e) {
            System.out.println("Error reading file");
        } finally {
            try {
                if(fr!=null) fr.close();
            } catch(IOException e) {}
            System.out.println("\nFile operation completed");
        }
    }
}

```

Execution Flow

try → attempt file operation

↓

Exception?

↓

Yes → catch executes

No → skip catch

↓

finally → always executes

5. Important Concepts

Multiple Catch Blocks

```

try {
    FileReader fr=new FileReader("data.txt");
} catch(FileNotFoundException e) {
    System.out.println("File missing");
} catch(IOException e) {
    System.out.println("General I/O error");
}

```

Order Matters

Specific exception → first

General exception → last

6. Real-World Example

Example: Log File System

Application writes logs into log.txt

Problems:

- File missing → FileNotFoundException
- Write failure → IOException

Solution:

try → write log

catch → handle error

finally → close file

Example: Student Record System

students.txt file

Operations:

- Read records
- Write records
- Update data

All must be handled safely using exceptions.

7. Best Practices

Practice	Explanation
Always close file	Avoid memory leak
Use finally block	Ensure closure
Handle specific exception	Better control
Avoid ignoring exceptions	Debugging issue

8. Modern Approach (Short Note)

Java provides **try-with-resources** (advanced):

import java.io.*;

```
class Test {  
    public static void main(String[] args) throws Exception {  
        try(FileReader fr=new FileReader("data.txt")) {  
            int i;  
            while((i=fr.read())!=-1) {  
                System.out.print((char)i);  
            }  
        }  
    }  
}
```

☞ Automatically closes file

Final Understanding

IOException → general file error

FileNotFoundException → file missing

try → risky code

catch → handle error

finally → always execute

Unit-7: Generics Concept

Generics is a feature in Java used to **write flexible, reusable, and type-safe code**. It allows you to **define classes, methods, and collections with a placeholder type**.

1. Purpose of Generics (Type Safety)

Definition: Generics allow you to **specify the type of data a class or method can handle**.

Core Idea: Generics = Use of type parameter (like <T>) instead of fixed type

Without Generics

```
import java.util.*;
class Test {
    public static void main(String[] args) {
        ArrayList list=new ArrayList();
        list.add("Hello");
        list.add(10);
        String s=(String)list.get(1);
        System.out.println(s);
    }
}
```

Problem

Runtime error → ClassCastException

With Generics

```
import java.util.*;
class Test {
    public static void main(String[] args) {
        ArrayList<String> list=new ArrayList<>();
        list.add("Hello");
        String s=list.get(0);
        System.out.println(s);
    }
}
```

Result

Compile-time safety ✓

No casting needed ✓

Key Understanding

Generics ensures correct data type at compile time

2. Problem Without Generics (Runtime Errors)

Issue

Without generics:

- Any type of object can be added
- Type checking happens **at runtime**
- Errors occur during execution

Example

```
ArrayList list=new ArrayList();
```

```
list.add("Java");  
list.add(100);  
String s=(String)list.get(1);
```

Runtime Error

ClassCastException

Why This Happens

Compiler does NOT know type → allows everything

Key Problem

Issue	Explanation
No type checking	Compile-time
Manual casting	Required
Runtime failure	Possible
Unsafe code	Yes

3. Generics Solution (Type Safety)

How Generics Solves Problem

```
ArrayList<String> list=new ArrayList<>();  
list.add("Java");  
// list.add(100); ✗ compile-time error
```

Benefit

Error detected at compile time ✓

Key Concept: <T> = Type parameter (placeholder for data type)

4. Advantages of Generics

1. Type Safety: Ensures only **correct type of data is stored**.

Example

```
ArrayList<Integer> list=new ArrayList<>();  
list.add(10);  
// list.add("Hello"); ✗
```

Benefit

Compile-time error instead of runtime error

2. No Need for Type Casting

Without Generics

```
String s=(String)list.get(0);
```

With Generics

```
String s=list.get(0);
```

Benefit

Cleaner and simpler code

3. Code Reusability

Concept

Write code once, use for multiple data types.

Example

```
class Box<T> {  
    T data;  
    void set(T data) {  
        this.data=data;  
    }  
    T get() {  
        return data;  
    }  
}
```

Usage

```
Box<Integer> b1=new Box<>();  
b1.set(10);  
Box<String> b2=new Box<>();  
b2.set("Hello");
```

Benefit: Same class works for different types

4. Compile-Time Checking

Concept: Errors are detected **before execution**

Benefit: Safer programs, fewer bugs

5. Better Performance

- No casting → faster execution
- Less runtime checking

5. Real-World Understanding

Example: Student Management

List<Student> → only student objects allowed

Example: Banking System

List<Transaction> → safe transaction handling

Example: E-Commerce

List<Product> → avoids wrong data type

6. Internal Working (Conceptual)

Generics → compile-time check

After compilation → type erased (Type Erasure)

Key Idea

Generics exist only at compile time

Final Understanding

Generics:

- ✓ Ensures type safety
 - ✓ Removes runtime errors
 - ✓ Avoids casting
 - ✓ Improves code reuse
- <T> → placeholder for data type

Unit-7: Defining Generic Classes

Generic classes allow you to **create a class that works with any data type** using a **type parameter** like <T>. They provide **type safety, flexibility, and reusability**.

1. Generic Class Syntax (<T> usage)

Definition: A **generic class** is a class that uses a **type parameter** (placeholder type) instead of a fixed data type.

Basic Syntax

```
class ClassName<T> {
    T variable;
    void set(T value) {
        variable=value;
    }
    T get() {
        return variable;
    }
}
```

Explanation

Part	Meaning
<T>	Type parameter (placeholder)
T variable	Variable of generic type
set()	Stores value
get()	Returns value

Example

```
class Box<T> {
    T data;
    void set(T data) {
        this.data=data;
    }
    T get() {
        return data;
    }
}

class Test {
    public static void main(String[] args) {
        Box<Integer> b1=new Box<>();
        b1.set(10);
        System.out.println(b1.get());
        Box<String> b2=new Box<>();
    }
}
```

```

        b2.set("Java");
        System.out.println(b2.get());
    }
}

```

Output

```

10
Java

```

Key Understanding

Same class works for different data types

2. Type Parameter (Placeholder Type)

Concept: A type parameter is a symbol (like T) used as a placeholder for actual data type.

Common Type Parameters

Symbol	Meaning
T	Type
E	Element
K	Key
V	Value
N	Number

Example

```

class Data<T> {
    T value;
}

```

How It Works

T → replaced with actual type during object creation

Usage

```

Data<Integer> d1=new Data<>();
Data<String> d2=new Data<>();

```

Key Insight

T is not a type, it becomes type later

3. Multiple Type Parameters (<K, V>)

Concept: A generic class can have **more than one type parameter**.

Syntax

```

class ClassName<K, V> {
    K key;
    V value;
}

```

Example

```

class Pair<K, V> {
    K key;
}

```

```

V value;
Pair(K key, V value) {
    this.key=key;
    this.value=value;
}
void display() {
    System.out.println(key+" : "+value);
}
}
class Test {
    public static void main(String[] args) {
        Pair<Integer,String> p=new Pair<>(1,"Java");
        p.display();
    }
}

```

Output

1 : Java

Real Understanding

K → Key

V → Value

Real Example

Student ID → Integer (K)

Student Name → String (V)

4. Internal Working (Conceptual)

Generic class defined → type parameter used

↓

Object created with specific type

↓

Compiler checks type safety

↓

No runtime casting needed

5. Advantages of Generic Classes

Advantage	Explanation
Type safety	Prevent wrong data type
Code reuse	Same class for multiple types
No casting	Cleaner code
Compile-time checking	Early error detection

6. Real-World Examples

1. Data Storage System

Box<T> → stores any type of data

2. Key-Value Mapping

Pair<K,V> → used in maps (HashMap)

3. Library System

Book<T> → different types of books

7. Important Rules

Rule	Explanation
Cannot use primitive types	Use wrapper classes
Type parameter in class name	Must be declared
Can use multiple types	<K, V>
Works at compile-time	Type checking

Example (Primitive Not Allowed)

Box<int> b; // ✗ not allowed

Box<Integer> b; // ✓ correct

8. Comparison: Normal Class vs Generic Class

Feature	Normal Class	Generic Class
Type	Fixed	Flexible
Reusability	Low	High
Type safety	Low	High
Casting	Required	Not required

Final Understanding

Generic class:

- ✓ Uses <T> as placeholder
- ✓ Works with any data type
- ✓ Ensures type safety
- ✓ Improves reusability

<T> → Type parameter

<K,V> → Multiple parameters

Unit-7: Generic Methods

Generic methods allow you to **define type parameters at the method level**, independent of the class. They provide **flexibility and type safety** for operations that can work on multiple data types.

1. Defining Generic Methods (Method-Level Generics)

Definition: A **generic method** is a method that introduces its own type parameter (like <T>) before the return type.

Basic Syntax

```
class Test {  
    <T> void display(T value) {  
        System.out.println(value);  
    }  
}
```

Key Structure

Part	Meaning
<T>	Type parameter (declared before return type)

T value	Parameter of generic type
Method body	Works for any type

Example

```
class Test {
    <T> void show(T data) {
        System.out.println(data);
    }
    public static void main(String[] args) {
        Test t=new Test();
        t.show(10);
        t.show("Java");
        t.show(3.14);
    }
}
```

Output

```
10
Java
3.14
```

Key Understanding

Same method works for multiple data types

2. Generic Method with Return Type

Concept: Generic methods can also return values of type T.

Example

```
class Test {
    <T> T getData(T data) {
        return data;
    }
    public static void main(String[] args) {
        Test t=new Test();
        int a=t.getData(10);
        String s=t.getData("Hello");
        System.out.println(a);
        System.out.println(s);
    }
}
```

Key Insight

Return type depends on input type

3. Type Inference (Automatic Type Detection)

Concept: Java automatically detects the type parameter based on the argument passed.

Example

```
class Test {
```

```

<T> void print(T data) {
    System.out.println(data);
}
public static void main(String[] args) {
    Test t=new Test();
    t.print(100);
    t.print("Java");
}
}

```

How It Works

Compiler sees:

print(100) → T = Integer

print("Java") → T = String

Explicit Type (Optional)

t.<Integer>print(100);

👉 Usually not required

Key Understanding

Type inference → compiler automatically decides type

4. Generic Method with Multiple Parameters

Concept: Generic methods can accept **multiple parameters of same type**.

Example

```

class Test {
    <T> void display(T a,T b) {
        System.out.println(a+" "+b);
    }
    public static void main(String[] args) {
        Test t=new Test();
        t.display(10,20);
        t.display("Hello","Java");
    }
}

```

Output

10 20

Hello Java

5. Generic Method with Different Types

Concept: Use **multiple type parameters**.

Example

```

class Test {
    <T,U> void show(T a,U b) {
        System.out.println(a+" "+b);
    }
}

```

```

    public static void main(String[] args) {
        Test t=new Test();
        t.show(10,"Java");
    }
}

```

Output

10 Java

6. Generic Methods vs Generic Classes

Comparison

Feature	Generic Method	Generic Class
Scope	Method level	Class level
Flexibility	High	Moderate
Type parameter	Defined in method	Defined in class
Use case	Utility functions	Data containers

Example Difference

Generic Class

```

class Box<T> {
    T data;
}

```

Generic Method

```

<T> void display(T data) {
}

```

7. Real-World Examples

1. Utility Methods

print(), swap(), search() → work for any type

2. Data Processing

Process Integer, String, Double using same method

3. Library Functions

Collections.sort(), Arrays.sort()

8. Advantages of Generic Methods

Advantage	Explanation
Reusability	One method for all types
Type safety	Compile-time checking
No casting	Cleaner code
Flexibility	Works with multiple types

9. Important Rules

Rule	Explanation
Declare <T> before return type	Mandatory
Can use multiple types	<T,U>
Cannot use primitives	Use wrapper classes

Works at compile time	Type safety
-----------------------	-------------

Example (Primitive Not Allowed)

```
t.print(10); // works as Integer
```

10. Internal Working

Method defined with <T>



Method called with value



Compiler assigns type



Type safety ensured

Final Understanding

Generic Method:

- ✓ Defined with <T> before return type
- ✓ Works with any data type
- ✓ Type decided automatically (type inference)

Unit-7: Generics with Collections

Collections + Generics give **type-safe, flexible, and efficient data structures** in Java.

Generics ensure that collections **store only a specific type of data**, preventing runtime errors.

1. Why Generics with Collections?

Problem Without Generics

```
import java.util.*;  
class Test {  
    public static void main(String[] args) {  
        ArrayList list=new ArrayList();  
        list.add("Java");  
        list.add(100);  
        String s=(String)list.get(1);  
    }  
}
```

Issue

Runtime error → ClassCastException

Solution Using Generics

```
ArrayList<String> list=new ArrayList<>();  
list.add("Java");  
// list.add(100); ✗ compile-time error
```

Core Idea

Generics + Collections → Type-safe data storage

2. ArrayList

Concept: ArrayList<T> is a **dynamic array** that stores elements of a specific type.

Syntax: ArrayList<Type> list=new ArrayList<>();

Example

```
import java.util.*;
class Test {
    public static void main(String[] args) {
        ArrayList<String> list=new ArrayList<>();
        list.add("Java");
        list.add("Python");
        list.add("C++");
        for(String s:list) {
            System.out.println(s);
        }
    }
}
```

Output

Java
Python
C++

Key Features

Feature	Explanation
Dynamic size	Grows automatically
Ordered	Maintains insertion order
Allows duplicates	Yes
Indexed access	Yes

Important Methods

Method	Purpose
add()	Insert element
get()	Access element
remove()	Delete element
size()	Number of elements

Key Insight

ArrayList<T> → Type-safe dynamic array

3. HashMap<K, V>

Concept: HashMap<K,V> stores data in **key-value pairs**.

Syntax: HashMap<Key, Value> map=new HashMap<>();

Example

```
import java.util.*;
class Test {
    public static void main(String[] args) {
        HashMap<Integer,String> map=new HashMap<>();
```

```

        map.put(1,"Java");
        map.put(2,"Python");
        map.put(3,"C++");
        for(Integer key:map.keySet()) {
            System.out.println(key+" "+map.get(key));
        }
    }
}

```

Output

```

1 Java
2 Python
3 C++

```

Key Features

Feature	Explanation
Key-value storage	Yes
Unique keys	No duplicates
Fast access	Hashing
Unordered	No fixed order

Important Methods

Method	Purpose
put()	Insert data
get()	Retrieve value
remove()	Delete entry
keySet()	Get all keys

Key Insight

HashMap<K,V> → Key-value mapping with type safety

4. LinkedList

Concept: LinkedList<T> is a **doubly linked list** that allows efficient insertion and deletion.

Syntax: LinkedList<Type> list=new LinkedList<>();

Example

```

import java.util.*;
class Test {
    public static void main(String[] args) {
        LinkedList<String> list=new LinkedList<>();
        list.add("Java");
        list.add("Python");
        list.addFirst("C++");
        list.addLast("Go");
        for(String s:list) {
            System.out.println(s);
        }
    }
}

```

```
}
```

Output

C++

Java

Python

Go

Key Features

Feature	Explanation
Dynamic structure	Yes
Fast insertion/deletion	Yes
Maintains order	Yes
Allows duplicates	Yes

Important Methods

Method	Purpose
add()	Add element
addFirst()	Insert at beginning
addLast()	Insert at end
remove()	Delete element

Key Insight: `LinkedList<T>` → Flexible list with fast updates

5. Comparison: ArrayList vs LinkedList vs HashMap

Feature	ArrayList	LinkedList	HashMap
Structure	Dynamic array	Linked list	Key-value
Access	Fast	Slower	Very fast
Insert/Delete	Slower	Faster	Fast
Order	Maintained	Maintained	Not maintained
Type	Single	Single	Key + Value

6. Real-World Examples

1. Student List

`ArrayList<Student>` → store students

2. User Data Mapping

`HashMap<Integer,User>` → ID → User

3. Task Management

`LinkedList<Task>` → dynamic task list

7. Why Generics are Important in Collections

Advantage	Explanation
Type safety	Prevent wrong data
No casting	Cleaner code
Compile-time check	Early error detection
Reusability	Generic design

Final Understanding

Collections + Generics:

ArrayList<T> → dynamic list

LinkedList<T> → flexible list

HashMap<K,V> → key-value mapping

Generics:

- ✓ Type safety
- ✓ No runtime errors
- ✓ Reusable code

Goal:

- ✓ Efficient data storage
- ✓ Safe data handling
- ✓ Flexible applications

Unit-7: Real-World Applications

This section connects concepts to **actual systems** where file handling and generics are used together to build **reliable, scalable, and reusable applications**.

1. File Processing Systems (Logs, Reports)

Concept: File processing systems **read, write, update, and analyze data stored in files**.

Where Used

- Log systems (system/app logs)
- Report generation (PDF/CSV/TXT)
- Data export/import (ETL)

Example: Log File System

Scenario: An application records events into log.txt.

Flow: Application event → write to file → store permanently → later analysis

Code (Writing Logs)

```
import java.io.*;
class LogWriter {
    public static void main(String[] args) throws Exception {
        BufferedWriter bw=new BufferedWriter(new FileWriter("log.txt",true));
        bw.write("User logged in");
        bw.newLine();
        bw.write("Transaction completed");
        bw.close();
    }
}
```

Code (Reading Logs)

```
import java.io.*;
class LogReader {
    public static void main(String[] args) throws Exception {
```



```

        BufferedReader br=new BufferedReader(new FileReader("log.txt"));
        String line;
        while((line=br.readLine())!=null) {
            System.out.println(line);
        }
        br.close();
    }
}

```

Why Important

Benefit	Explanation
Permanent record	Store history
Debugging	Identify errors
Monitoring	Track activity

Example: Report Generation

Student data → processed → saved as report.txt

Code

```

import java.io.*;
class Report {
    public static void main(String[] args) throws Exception {
        BufferedWriter bw=new BufferedWriter(new FileWriter("report.txt"));
        bw.write("Student Report");
        bw.newLine();
        bw.write("Marks: 85");
        bw.close();
    }
}

```

2. Data Storage Systems (Databases, Records)

Concept: Files are used to store structured data such as records, transactions, and user data.

Example: Student Record System

students.txt

ID Name Marks

1 Dhruv 90

2 Amit 85

Code (Store Data)

```

import java.io.*;
class StudentData {
    public static void main(String[] args) throws Exception {
        BufferedWriter bw=new BufferedWriter(new FileWriter("students.txt",true));
        bw.write("1 Dhruv 90");
        bw.newLine();
        bw.write("2 Amit 85");
        bw.close();
    }
}

```

```
}
```

Code (Read Data)

```
import java.io.*;
class ReadData {
    public static void main(String[] args) throws Exception {
        BufferedReader br=new BufferedReader(new FileReader("students.txt"));
        String line;
        while((line=br.readLine())!=null) {
            System.out.println(line);
        }
        br.close();
    }
}
```

Real Systems

System	Usage
Banking	Store transactions
E-commerce	Store orders
Healthcare	Store patient records
Education	Store student data

Important Insight

Files act as simple database storage systems

3. Generics Usage (Reusable Libraries)

Concept: Generics are used to create **reusable, flexible, and type-safe components**.

Example: Generic Data Container

```
class Box<T> {
    T data;
    void set(T data) {
        this.data=data;
    }
    T get() {
        return data;
    }
}
class Test {
    public static void main(String[] args) {
        Box<Integer> b1=new Box<>();
        b1.set(10);
        System.out.println(b1.get());
        Box<String> b2=new Box<>();
        b2.set("Java");
        System.out.println(b2.get());
    }
}
```

Why Important

Benefit	Explanation
Reusability	Same class for all types
Type safety	Prevent wrong data
Maintainability	Clean design

Example: Generic Utility Method

```
class Utility {  
    static <T> void printArray(T[] arr) {  
        for(T i:arr) {  
            System.out.println(i);  
        }  
    }  
    public static void main(String[] args) {  
        Integer a[]={1,2,3};  
        String b[]{"Java","Python"};  
        printArray(a);  
        printArray(b);  
    }  
}
```

Real-World Use

Area	Example
Collections	ArrayList, HashMap<K,V>
Libraries	Reusable APIs
Frameworks	Spring, Hibernate
Data handling	Generic data models

4. Combined System (I/O + Generics)

Example: Student Management System

File → stores student data

Generics → manage data in memory

Flow

File → read data → store in ArrayList<Student> → process → write back

Code Concept

```
import java.util.*;  
class Student {  
    int id;  
    String name;  
    Student(int id,String name) {  
        this.id=id;  
        this.name=name;  
    }  
}  
  
class Test {  
    public static void main(String[] args) {  
        ArrayList<Student> list=new ArrayList<>();  
    }  
}
```

```

        list.add(new Student(1,"Dhruv"));
        for(Student s:list) {
            System.out.println(s.id+" "+s.name);
        }
    }
}

```

5. Key Advantages in Real Systems

Feature	Benefit
File handling	Persistent storage
Buffered streams	Efficient I/O
Generics	Type safety
Collections	Easy data management

6. Industry-Level Understanding

Modern systems use:

File systems → logs, reports

Databases → structured data

Generics → reusable code

Collections → data management

Final Understanding

File Processing:

- ✓ Logs, reports, records

Data Storage:

- ✓ Files, databases

Generics:

- ✓ Reusable, type-safe code

Combined:

- ✓ Efficient systems
- ✓ Safe data handling
- ✓ Scalable applications